Project leader
Project Leader: Claire
Iteration 2: System design and Implementation Start

Responsibilities:

- Coordinate progress between team members
- Ensure tasks are divided equally
- Keep the task table updates
- Present the teams progress during the check-in meeting
- Ensure the program compiles and runs successfully

Team Responsibilities:

- Claire: Soil moisture simulation, rainfall simulation, CSV output, and water usage calculations.
- Nyesha: Timer controller, smart controller, text-based menu, and settings.
- Both: System design, testing, GitHub updates, and documentation.

Classes

Class 1: GardenSimulation

Owner: Claire

Private data:

- double soilMoisture
- bool rainExpected
- int simulationDays

Public methods:

- simulateSoilMoisture()
- simulateRainfall()
- getSoilMoisture()
- isRainExpected()

Class 2: IrrigationController

Owner: Nyesha

Private data:

- double moistureThreshold
- int wateringInterval
- double waterPerSession

Public methods:

- runTimerController()
- runSmartController()

- changeSettings()

Class 3: ResultsManager

Owner: Claire

Private data:

- double timerWaterUsed
- double smartWaterUsed

Public methods:

- compareWaterUsage()
- saveResultsToCSV()

Main menu design

Smart Irrigation Controller
1. Run soil moisture simulation
2. Run rainfall simulation
3. Run timer controller
4. Run smart controller
5. Compare water usage
6. Save results to CSV
7. Change settings

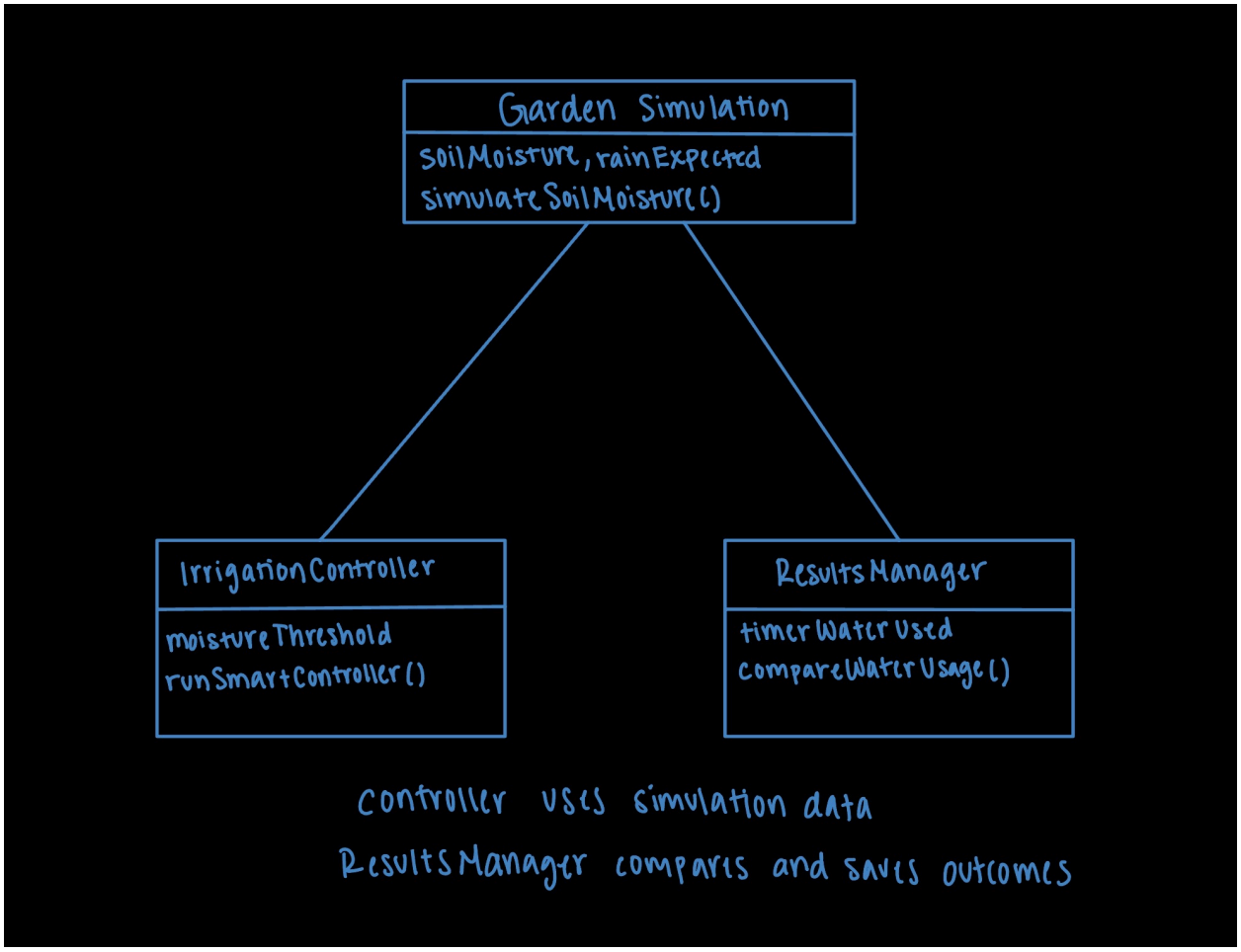
0. Exit

- The user enters a number
- A switch statement selects the corresponding task
- Each case calls one function
- Unfinished features print “Not implemented yet.”
- Option 0 exits the program
- Invalid input display an error message

File structure

main.cpp : menu and program entry point
simulation.h : GardenSimulation class declaration
simulation.cpp : Simulation implementation
controller.h : IrrigationController class declaration
controller.cpp : controller implementation
results.h : ResultsManager class declaration
results.cpp : Results and CSV implementation
README.md : build and run instructions
docs/ : PDF report and excel project plan

Class diagram:



Implementation Process - Nyesha

- Created a working C++ program with a text-based menu using a switch statement.
- Implemented the timer controller to simulate watering at fixed intervals.
- The timer controller waters every 2 days and uses 5 liters per watering session.
- Created the IrrigationController class with private variables and public methods.
- Created the GardenSimulation and ResultsManager class structures.
- Added stub functions for unfinished features that display "Not implemented yet."
- Organized the program into separate header (.h) and source (.cpp) files.
- Successfully compiled the program using g++ with C++17.
- Uploaded the source code to the team's GitHub repository.
- Updated the README with compilation and execution instructions.

Testing Results - Nyesha

- Test 1: Main Menu
    - Verified that all menu options appear correctly.
    - Confirmed that the switch statement calls the corresponding functions.
    - Result: Passed.

- Test 2: Timer Controller
    - Entered 10 simulation days.
    - Program watered on days 2, 4, 6, 8, and 10.
    - Total water usage was 25 liters.
    - Result: Passed.
- Test 3: Invalid Input
    - Entered 9, which displayed "Invalid choice. Try again."
    - Entered "abc", which displayed "Invalid input. Enter a number."
    - Result: Passed.
- Test 4: Unfinished Features
    - Selected the smart controller and settings options.
    - Both displayed "Not implemented yet."
    - Result: Passed.
- Test 5: Exit
    - Selected option 0.
    - Program displayed "Exiting program..." and terminated.
    - Result: Passed.

Implementation Process - Claire

- Implemented the rainfall simulation in the GardenSimulation class. The user enters the number of days (1 to 365), and each day has a 25% chance of 2 to 15 mm of rain.
- Used a fixed random seed (srand(2140)) so every run produces the same rainfall, allowing both controllers to be compared on the same weather.
- Added a rain forecast that shows whether rain is expected the next day.
- Implemented the soil moisture simulation. Moisture starts at 50%, drops 6% each day, rises 1% per mm of rain, and stays between 0% and 100%. Days below 30% are marked "Dry."
- Added the nextMoisture() function and daily getter functions so the smart controller can use the simulation data.
- Implemented the water usage comparison in the ResultsManager class, which shows each controller's total water use, liters saved, and percent saved.
- Implemented CSV output, which saves the water totals and daily rainfall, forecast, and soil moisture to irrigation_results.csv.
- Completed the results and CSV tasks ahead of schedule (planned for Iteration 3).
- Successfully compiled the program using g++ with C++17 and no warnings.
- Pushed the code to the team's GitHub repository and updated the README.

Class Updates - Claire

- GardenSimulation private data added: int rainfall[MAX_DAYS], double moisture[MAX_DAYS], bool hasData
- GardenSimulation public methods added: getSimulationDays(), getRainfallOnDay(), isRainExpectedOnDay(), getMoistureOnDay(), nextMoisture()
- ResultsManager private data added: GardenSimulation* simulation
- ResultsManager public methods added: setTimerWaterUsed(), setSmartWaterUsed(), setSimulation()

Testing Results - Claire

- Test 1: Rainfall Simulation
  - Entered 30 simulation days.
  - 5 of 30 days had rain, with 40 mm of rain in total.
  - The forecast showed "Yes" on the day before each rainy day.
  - Result: Passed.
- Test 2: Soil Moisture Simulation
  - Selected the soil moisture simulation before running rainfall. The program ran the rainfall simulation first automatically.
  - Moisture started at 50% and dropped 6% on each day without rain.
  - Without watering, 24 of 30 days were dry (below 30%).
  - Result: Passed.
- Test 3: Invalid Input
  - Entered "abc", 0, and 400 as the number of days.
  - Each displayed "Invalid number of days. Enter 1 to 365."
  - Result: Passed.
- Test 4: Compare Water Usage
  - Before the controllers ran, the program displayed "Run both the timer controller (3) and smart controller (4) first."
  - Using a test program with totals of 75 liters (timer) and 40 liters (smart), it displayed "Water saved: 35 liters (46%)."
  - Result: Passed.
- Test 5: Save Results to CSV
  - Created irrigation_results.csv with the water totals and 30 rows of daily data.
  - Result: Passed.
- Test 6: Timer Controller After Simulations
  - Ran the timer controller for 10 days after running both simulations.
  - It still showed 25 liters total, so the new code did not affect existing features.
  - Result: Passed.

Contribution Table

Team Member	Contributions	Status
Nyesha	Created main menu, timer controller, controller class, simulation/results class stubs, updated README, tested program, pushed code to GitHub	Completed
Claire	Project leader; implemented soil moisture and rainfall simulations, water usage comparison, and CSV output; tested features; updated README; pushed code to GitHub	Completed (results and CSV finished ahead of schedule)